SWARMA

Подписи и аутентификация в Swarma

От А до Я: путь команды, ключи и границы доверия

25 октября 2026 года

Для оператора и команды.

Ключи, полномочия и проверка перед действием.

Содержание

0 Зачем подписывать . . . 3
0.1 Основания . . . 3

1 Кто держит ключи . . . 4

2 Критическая команда с телефона . . . 5

3 Обычное сообщение из Element . . . 6
3.1 Почему не подписывать каждую реплику . . . 6
3.2 Переходный режим . . . 6

4 Агент подписывает агенту . . . 7

5 Новое и потерянное устройство . . . 8

6 Реестр владельца и свежесть . . . 9
6.1 Журнал и контрольные точки . . . 9
6.2 Свежесть снимка . . . 9

7 Когда проверка отказывает . . . 10

8 Что находится в конверте . . . 11

9 Команда через два сервера и два владельца 12
9.1 Доверие к другому владельцу . . . 12

10 Уверенность и право действовать . . . 13
10.1 Как проходит проверка гранта . . . 13

11 Как устроен Signer . . . 14
11.1 Открытые вопросы . . . 14

0 Зачем подписывать

Аутентификация устанавливает отправителя. **Авторизация** решает, разрешено ли ему действие. Подпись связывает точные байты с ключом, а грант ограничивает действие, цель и условия. Шифрование скрывает содержимое от посторонних и прав не выдаёт.

Уровень уверенности описывает, чем подтверждено происхождение сообщения. Уровней четыре:

1. При **A0** отправителя утверждает homeserver носителя. Захваченный сервер или учётная запись выдают команду от чужого имени, поэтому имя на экране и членство в комнате происхождения не доказывают.
2. При **A1** получатель проверил E2EE-устройство человека или подпись аттестованного ключа агента. Содержимое агентского сообщения остаётся данными коллеги.
3. При **A2** ключ человека подписывает точное содержимое, и потребитель проверяет его текущее полномочие на конкретное действие и цель. Подпись сама не доказывает, что человек видел текст.
4. При **A3** квалифицированный провайдер подписи показывает человеку точное действие, требует присутствия при каждом использовании ключа и связывает подтверждение с подписанными байтами.

Пример. Сообщение “удали черновик” под знакомым именем удаления не разрешает. Правильная подпись тоже не поможет, если грант разрешает только чтение. Пересказ агента “владелец одобрил” остаётся данными.

0.1 Основания

Swarm Signer объединяет аппаратный корень оператора, аттестации устройств и общее ядро проверки. Мост с воротами доверия проверяет сообщение, ключник подписывает за агента, реестр владельца хранит и публикует подписанные записи, а исполнитель проверяет полномочие на границе эффекта.

Текст опирается на книгу Swarma 1.3 (главы “Ключи и ключник”, “Гранты и полномочия”, “Путь сообщения и мост”, “Допуск к носителю”, “Идентичность агента”), на схему Signer от 23-24 октября 2026 года и на три спецификации модулей: формат подписанных записей платформы `swarm-envelope/2` (срез v0), дизайн службы реестра владельца и спецификацию ключника. Формат ячеек Swarm Chain эта схема не меняет. Где текст расходится со спецификацией модуля, права спецификация.

1 Кто держит ключи

Аттестация связывает публичный ключ с владельцем, классом, назначением и сроком. Её подписывает закреплённый корень нужной роли, а сам корень закрепляют доверенным способом вне чата. Корень аттестаций ключника, корень реестра, корень оператора и корень грантов попарно различны, и совпадение ролей отклоняется до проверки подписи. Корнем оператора владелец аттестует свои устройства и аккаунты носителя.

Ключ	Где и у кого	Что подписывает	Как прекращается доверие
Корень оператора	у владельца; в схеме Signer аппаратный FIDO-ключ	аттестации устройств и аккаунтов, снимки отзывов своего домена	заменяет и отзывает его только средство восстановления
Ключ устройства	Secure Enclave iPhone или Mac; **P-256 с low-S**	действия оператора, одобрения, делегированные гранты	отзыв корнем оператора, истечение аттестации
Корень аттестаций ключника	ключник	только аттестации агентов сроком до 24 часов	смена корня с перевыпуском аттестаций
Ключ агента	ключник, **Ed25519**; у агента секрета нет	сообщения агента `swarma-msg`	отзыв привязки, истечение аттестации
Корень и сервис грантов	у владельца; сервис (Ed25519) у отдельной службы	гранты; сервис только под родительским грантом	снимок отзывов домена грантов
Ключ реестра	изолированный процесс подписи на узле доверия	снимки отзывов агентов, контрольные точки, записи свежести	смена закрепления с сохранением непрерывности
Matrix master, self-signing, device; OMEMO device	E2EE-клиент	цепочку cross-signing и обмен E2EE	отзыв аттестации master или fingerprint



Рис. 1.1. От закреплённых корней к ключам разных назначений

Компрометация ключа ограничена его ролью. Украв ключ реестра, нападающий может задержать или скрыть публикацию, но гранта или аттестации не выпустит. Отзыв верхнего звена закрывает доверие ко всей зависимой ветке. Корень оператора заменяет или отзывает только отдельно хранимое, заранее аттестованное средство восстановления, а следующая форма средства требует согласия двух хранителей из трёх.

2 Критическая команда с телефона

Критическое действие оператор подтверждает на телефоне записью **`operator-act/1`**. Swarm Signer показывает операцию, конкретную цель, владельца, адресата, аргументы и срок, и человек видит их до подписи.

Провайдер ключа платформы требует присутствия через Face ID и подписывает ключом **P-256** из Secure Enclave. Ключ из Secure Enclave не выходит, ядро получает только подпись. Подпись сырая: 64 байта `r || s` с low-S над полным подписываемым представлением записи. Утверждение FIDO/WebAuthn подписывает другие байты (`authenticatorData` и хэш `clientDataJSON`), поэтому как подпись действия оно отклоняется, даже если в вызове лежит то же представление.

Запись несёт операцию `operation`, одну типизированную цель `target`, владельца `owner`, эпоху `epoch`, одноразовый `action_id`, дайджест аргументов `arguments_digest` и флаг `confirmed`. Срок жизни записи не больше 120 секунд.

Ворота доверия проверяют подпись и аттестацию устройства от закреплённого корня оператора, текущий статус ключа, свежий снимок отзывов и совпадение владельца в аттестации, в записи и в регистрации. Затем исполнитель атомарно резервирует `action_id` и только после этого выполняет эффект. Повторное использование того же подтверждения отклоняется (`act_replay`), даже если байты совпадают. Квитанция доставки исполнения не подтверждает.

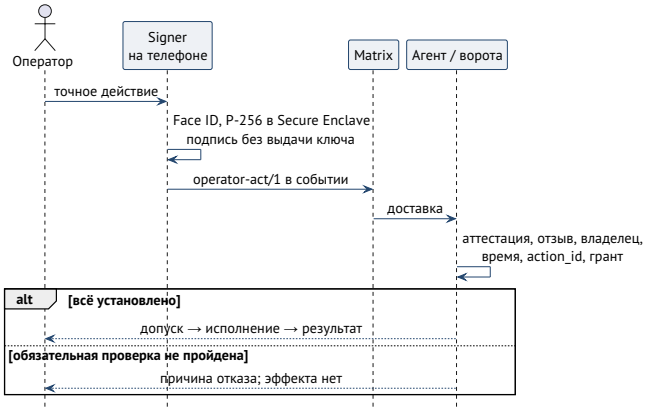


Рис. 2.1. Подтверждение точного действия и проверка до эффекта

Сужение и расширение. Политика потребителя делит операции на сужающие и расширяющие, отправитель класс не выбирает. Сужающее действие, например постоянный отзыв ключа агента, проходит с текущим аттестованным ключом устройства и уровня A3 не заявляет. Расширяющее действие (выдача грантов, привязки, допуски) требует A3: `confirmed=true` и действующее свидетельство провайдера о том, что ключ требует присутствия при каждом использовании и показывает точные операцию, цель, владельца, эпоху, адресата, `action_id`, аргументы и сроки.

Квалификация A3. Уровень A3 даёт только квалифицированный провайдер. Приложение подписи и провайдер платформы проходят ревью по письменной модели угроз, с аппаратной аттестацией там, где платформа её даёт. Первым квалифицируется один ключ P-256 в Secure Enclave телефона на iOS. Флаг `confirmed` и программная подпись сами по себе A3 не устанавливают, и без квалифицированного провайдера расширяющее действие отклоняется (`act_a3_inactive`).

3 Обычное сообщение из Element

Обычная реплика из Element идёт без конверта Signer. Оператор пишет с проверенного устройства в E2EE-комнате, и получатель устанавливает происхождение по нативной цепочке Matrix.

Цепочка проверяется в четыре звена: **device ← self-signing ← master ← аттестация корня оператора**. Ключ self-signing подписывает устройство, master аккаунта подписывает self-signing, а корень оператора аттестует master. Знакомого display name для этого недостаточно. Если цепочка сходится до закреплённого корня, доверие актуально и полная учётная запись совпадает с отправителем, адаптер сообщает **A1**.

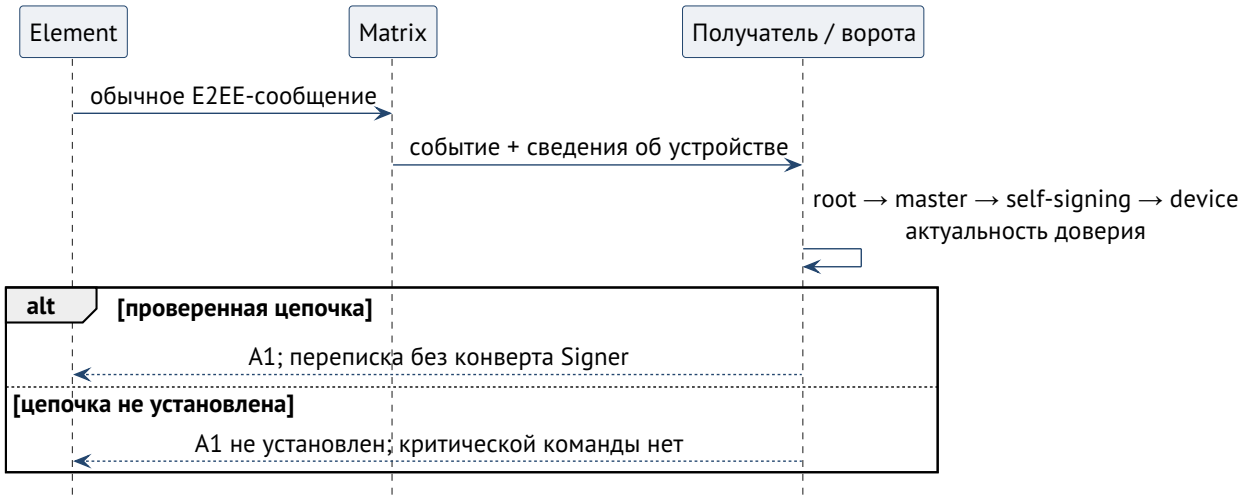


Рис. 3.1. E2EE даёт A1 без отдельного конверта Signer

Сообщение уровня A1 годится для переписки. Критическую команду оно не подтверждает и нового гранта не выдаёт: для этого оператор переходит в Signer. Непроверенное устройство A1 не получает, даже если аккаунт знаком, и отсутствие цепочки не превращает реплику в допустимую команду уровня A0.

3.1 Почему не подписывать каждую реплику

Для обсуждения достаточно знать, что собеседник пишет с проверенного устройства. Нативная цепочка E2EE избавляет от биометрии на каждое сообщение. Сервер не может добавить доверенное устройство без нужных подписей, если закреплённые ключи и проверяющий клиент не скомпрометированы.

3.2 Переходный режим

В переходном режиме команды A0 и A1 допускаются только от заранее названных людей в их записанных зонах. Оператор задаёт зоны вне области записи агента, из чата их не узнают. Когда человеческие команды переходят на A2, зоны заменяются грантами, а A0 остаётся данными.

Канал владельца привязывают при подъёме станции: допуск к носителю, затем провайдер моста, затем личная комната. Владельца не назначает первый написавший. Привязка комнаты и приглашение криптографической проверки и гранта не заменяют.

4 Агент подписывает агенту

Агент готовит тело `message/1` и подписываемое представление: метка `swarma-msg/2`, байт `0x0a` и CCJ-1 подписанной записи. Ключник узнаёт UID вызывающего от ядра ОС и находит его активную привязку и текущую аттестацию; имя в запросе личность не выбирает. Перед подписью ключник сверяет ровно три пути: `key` и `body.author` равны ключу агента, `body.author_attestation` равна идентификатору текущей аттестации. Затем он подписывает исходные байты ключом **Ed25519** и возвращает подпись, а закрытый ключ агенту и мосту не передаётся.

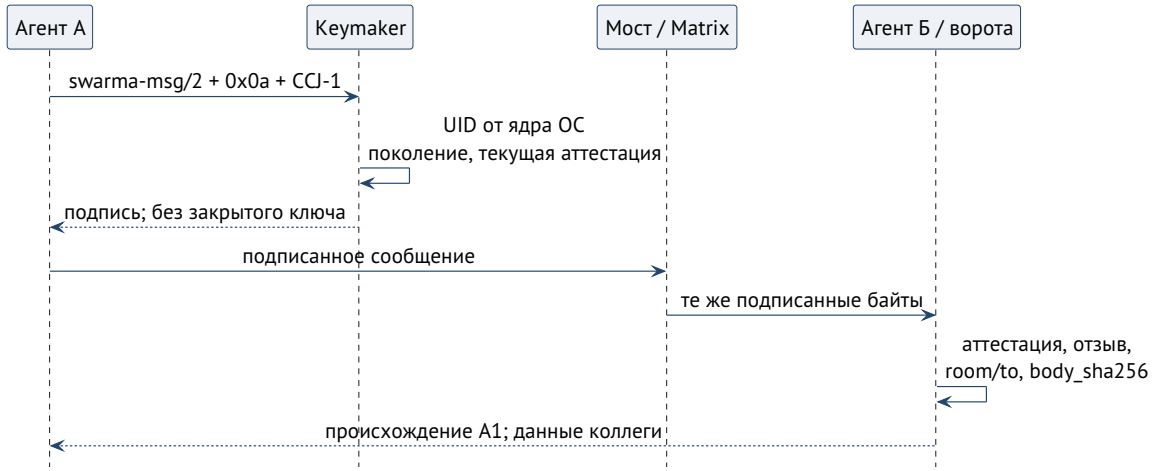


Рис. 4.1. Ключник подписывает за установленную ядром ОС личность

Аттестация агента (`attestation/1`, класс `agent`, назначение `swarma-msg`) подписана корнем аттестаций ключника. Её привязка точно повторяет `uid`, `generation`, `registration_number`, `master_epoch`, `owner` и `path` активного поколения. Срок не больше 86 400 секунд (24 часа), и предел проверяют обе стороны.

Ключник обновляет аттестацию сам, до истечения прежней. Новая аттестация становится текущей только вместе с **доказательством публикации**: записью в журнале реестра, покрывающей подписанной контрольной точкой и подтверждением свидетеля отката, сохранёнными в одной транзакции. С начала обновления и до сохранения доказательства ключник подписи не выдаёт (`key_not_enrolled`) и к прежней аттестации не возвращается. Истёкшая аттестация закрывает подпись.

Получатель берёт аттестацию по идентификатору `author_attestation` из подписанного тела и проверяет подпись корня, текущую привязку, срок и снимок отзывов, затем совпадение ключа с `author`. Дальше идут `room` и `to`, хэш текста `body_sha256`, окно `iat` и `exp` не больше 120 секунд и журнал идентификаторов. Ссылка на точную аттестацию не даёт подставить ключ из другого поколения.

Подтверждённое происхождение даёт **A1**: получатель видит данные коллеги, и подпись не одобряет сессию. Подписанный отчёт связывается с конкретным агентом и проверяемо неизменен, а приписка “оператор разрешил удалить данные” остаётся утверждением автора. Для разрешения нужно собственное проверяемое основание оператора.

5 Новое и потерянное устройство

Включить новое устройство

1. Оператор создаёт ключ **P-256** в Secure Enclave нового iPhone или Mac. Секрет остаётся за провайдером ключа платформы, ядро получает только публичный ключ.
2. Оператор сверяет публичный ключ доверенным способом. Корень оператора подписывает аттестацию: класс `operator-device`, роли `act` и `approver`, действия `act.sign`, `grant.sign` и `message.approve`, привязку к владельцу и обязательный срок.
3. Реестр владельца записывает аттестацию в журнал и публикует новую голову. Подпись корня остаётся исходной: реестр хранит и упорядочивает записи, но содержимое не переподписывает.
4. Устройство подписывает действия только после того, как проверяющий установил цепочку до корня, текущий статус ключа и грант на конкретное действие. Вход в приложение аттестацией не считается.

Для Matrix корень оператора аттестует master аккаунта, а устройства проверяются через self-signing. Для XMPP без cross-signing корень аттестует каждый OMEMO fingerprint отдельно. Первое закрепление корня проходит доверенную церемонию с доказательством владения новым ключом.

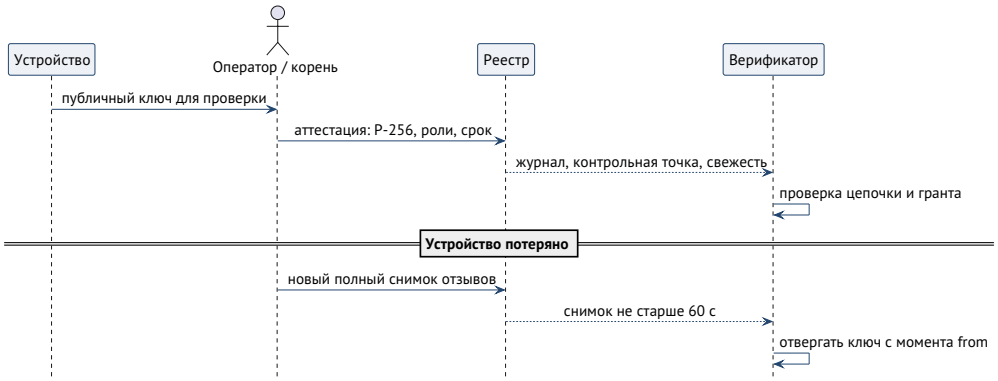


Рис. 5.1. Аттестация добавляет доверие, отзыв прекращает его

Потерять и отозвать

1. Оператор подписывает корнем оператора новый полный снимок отзывов `revocation/1`. В нём остаются все прежние записи и появляется запись `{kind, id, from}` об утраченном ключе.
2. Реестр публикует снимок. Пусть S означает время выпуска первого снимка с этой записью, а A означает максимальный возраст снимка, не больше 60 секунд. Каждый проверяющий отвергает ключ не позже S + A. При будущем `from` ключ отвергается не раньше `from` и не позже большего из S + A и `from`.
3. Оператор доводит отзыв до завершения. Отзыв завершён, когда названный набор потребителей подтвердил его через надёжные барьеры или когда истекли все ранее допущенные полномочия. После завершения окна допуска для отозванного права не остаётся.

Снимки накопительные: запись нельзя убрать, сдвинуть её `from` позже или выпустить очищающий снимок. Старое заявленное `iat` действующий отзыв не обходит. Критическое действие, выбранное политикой потребителя, дополнительно спрашивает владельца полномочия онлайн прямо перед эффектом, и недоступная проверка отклоняет действие.

6 Реестр владельца и свежесть

Реестр владельца служит единственным авторитетным писателем ключей, аттестаций, грантов, отзывов и журналов личностей одного владельца. Он хранит, упорядочивает и публикует записи и сохраняет подписи их исходных авторов. Реестр работает на отдельно администрируемом узле доверия, куда у платформы агентов нет административного доступа, а у агентов root-доступа.

Ключ реестра подписывает записи трёх видов: снимки отзывов `revocation/1` в своей области, контрольные точки журнала `checkpoint/1` и записи свежести `freshness/1`. Этот ключ держит изолированный процесс подписи под отдельной учёткой, и процесс подписывает только заранее одобренные профили записей. Компрометация узла достигает и этого процесса, но остальные корни живут вне реестра.

6.1 Журнал и контрольные точки

Каждое изменение головы реестр фиксирует неподписанной записью журнала `journal-entry/1`. Она несёт номер `seq`, ссылку на предыдущую запись `prev`, время `at`, операцию `add` или `revoke`, тип и идентификатор исходной записи. Подписанная контрольная точка `checkpoint/1` называет голову журнала `head`, её номер и текущий снимок отзывов реестра `snapshot`. Git служит только хранилищем и транспортом.

Свидетель отката работает на том же узле в отдельной учётке и процессе, и писатель реестра в его поток не пишет. Доверенная контрольная точка свидетеля хранится вне узла. Свидетель подтверждает намерение и совпадающее зафиксированное разрешение каждой транзакции, и реестр публикует голову только после этого. Тот же номер с другой головой или контрольной точкой означает раздвоение истории (`journal_fork`), меньший номер означает откат (`journal_rollback`). Реестр и свидетель на одном узле образуют один домен отказа: потеря узла закрывает допуск у потребителей.

6.2 Свежесть снимка

Потребитель принимает снимок отзывов не старше 60 секунд, а его политика может требовать меньшего возраста. Неизменный снимок реестр не переиздаёт. Каждые 20 секунд и сразу после изменения головы он публикует подписанную **запись свежести** `freshness/1` с меткой `swarma-registry freshness/1`. Запись называет неизменный снимок и текущую контрольную точку. Её глобальный номер `publication_seq` растёт ровно на единицу, `prev` ссылается на предыдущую запись той же головы, а время `issued_at` строго растёт.

Снимок старше 60 секунд проходит только полную проверку свежести. Последняя запись свежести должна быть не старше предела, её ссылки должны совпадать со снимком и контрольной точкой, а свежее аутентифицированное чтение головы свидетеля должно совпадать с этой контрольной точкой. Потребитель хранит последнюю принятую позицию и отвергает пропуск, откат и раздвоение публикаций. Изменённый снимок сначала проходит журнал и новую контрольную точку, и старая запись свежести его не продлевает.

Свежесть не возвращает истёкших грантов и не доказывает завершения отзыва. Связанная история доказывает полноту только от сохранённой позиции до предъявленной.

7 Когда проверка отказывает

Проверка работает по правилу fail closed: если обязательное свидетельство не установлено, нового разрешения и эффекта нет. Ворота доверия проходят проверки в фиксированном порядке и возвращают код первой неудачи с пустым планом. Оператор видит причину, а аудит сохраняет отказ.

Условие	Код отказа	Что проверить перед повтором
Снимок отзывов недоступен, устарел или откатился	`revocation_unavailable`; у сообщения `key_binding_unavailable`	доступность реестра, запись свежести и голову свидетеля; старый снимок проверку не обходит
Ключ или аттестация отозваны	`revoked`	новую аттестацию или другое разрешённое устройство
Срок записи истёк или не начался	`expired`, `not_yet_valid`	часы и время подписи; повтор требует новой подписи
Другая комната или адресат	`message_room_mismatch`, `message_recipient_mismatch`	полные ID комнаты и адресата; пересылка команду не переназначает
Тот же `id` с другими байтами	`envelope_conflict`	исходную отправку; повтор отправляет те же байты
Подтверждение действия уже использовано	`act_replay`	результат прежнего эффекта в журнале исполнителя
Грант не покрывает действие и цель	`grant_not_authorized`	цепочку грантов и требуемый уровень; подпись прав не добавляет
Расширение без квалифицированного провайдера A3	`act_a3_inactive`	квалификацию провайдера; сужающие действия проходят

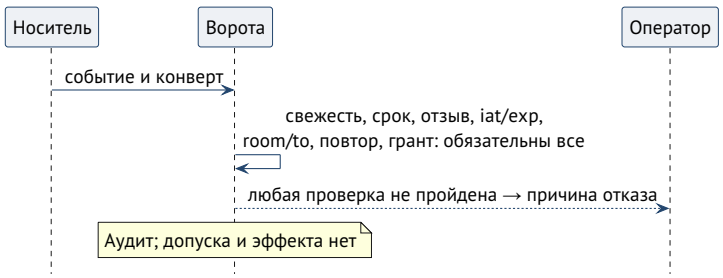


Рис. 7.1. Любой обязательный отказ останавливает допуск до эффекта

Неизвестный ключ, неверная подпись и несовпадение хэша доставленного текста тоже закрывают допуск. Неверную подпись нельзя обойти, доставив тело как пользовательскую команду: неподписанная переписка остаётся данными. Частичного действия ради проверки нет.

После устранения причины ворота проверяют все условия заново. При неясном исходе исполнитель сначала выясняет результат прежнего эффекта по своему журналу: новое подтверждение с новым `action_id` не защищает от повторного внешнего действия.

Экстренная остановка не зависит от доступности реестра и свежих снимков. Перезапуск после остановки требует отдельного текущего полномочия.

8 Что находится в конверте

Формат конверта **`swarm-envelope/2`**, каноническая форма тела **CCJ-1**. В Matrix готовый конверт лежит в содержимом события под ключом **`works.relux.swarm.envelope`** без изменения байтов. Конверт состоит ровно из четырёх членов: `version`, `body`, `id` и списка `signatures` из 1-8 подписей вида `{role, key, sig}`.

Тело сообщения `message/1` закрыто, и лишний или пропущенный член отклоняется. Поле `aud` называет одну точную службу-получателя, `author` и `author_attestation` называют ключ агента и его точную аттестацию, а `action`, `target` и `arguments` задают зарегистрированное действие, типизированную цель и аргументы для схемы обработчика. Поля `room` и `to` несут полные ID комнаты и адресата, `body_sha256` несёт SHA-256 точных UTF-8 байтов текста события, а `iat`, `exp` и `nonce` задают окно не больше 120 секунд и 16 случайных байтов.

Учебный пример. Текст события: `Отчёт готов, проверки пройдены.` в UTF-8 без завершающего перевода строки. Ключ и аттестация взяты из синтетических тестовых данных формата, `nonce` задан вручную, `body_sha256` и `id` вычислены, подпись заменена заглушкой; вектором совместимости пример не служит.

```
{"version": "swarm-envelope/2",
 "body": {"type": "message/1", "aud": "bridge-beta",
  "author": "ed25519:wue2t1GJeLZTqzYudkzRIhyY11lAzyPXDjsTXgXoIrk",
  "author_attestation":
    "sha256:c3cfbd2085a31b0b23e841afc84fa28d4e286e198a60021bcf02ec46251b1090",
  "action": "message.deliver", "target": {"kind": "station", "id": "beta/worker"},
  "arguments": {}, "room": "!review:beta.example", "to": "@worker:beta.example",
  "body_sha256": "1fb1adcbb22726f298ad56d77e627215da37a5c5ecc84ae785b920db13d316e5",
  "iat": "2026-10-25T09:00:00Z", "exp": "2026-10-25T09:02:00Z",
  "nonce": "ZXhhbXBsZS1ub25jZS0xNg"},
 "id": "sha256:de73e79d268edce55038729950553bab46350d9e2df03fb304ad7df534ab71e2",
 "signatures": [{"role": "author", "sig": "EXAMPLE_NOT_A_SIGNATURE",
   "key": "ed25519:wue2t1GJeLZTqzYudkzRIhyY11lAzyPXDjsTXgXoIrk"}]}
```

Отступы служат для чтения, проверка идёт по каноническим байтам. Действие `message.deliver` разрешает только доставку на станцию и исполнения текста не разрешает.

Идентификатор конверта считается по метке, байту `0x0a` и CCJ-1 тела: `id = sha256(метка || 0x0a || CCJ-1(body))`. Подпись покрывает другое представление: метку, `0x0a` и CCJ-1 подписанной записи `{body, key, role, version}`. Поэтому подпись одной роли, ключа, версии или типа не проверяется после замены любого из них. Завершающего перевода строки нет ни там, ни там.

Метка состоит из 1-96 байтов ASCII от `0x20` до `0x7e`. Метки берутся из закрытого списка попарно различных значений: `swarma-msg/2` для сообщений и отдельные метки для действий оператора, грантов, аттестаций, отзывов, записей регистратора, журнала, контрольных точек и свежести. Проверяющий сравнивает метку побайтно до первого `0x0a`, без нормализации. Разделитель различает представления разных меток, даже когда одна служит префиксом другой.

Ключи и подписи. Ключ агента `ed25519:` несёт 32 байта, ключ устройства `p256:` несёт 33 байта сжатой точки SEC1 и подписывает сырыми `r || s` с low-S. Подпись занимает 64 байта в base64url, а DER, high-S и обёртки SSHSIG и FIDO отклоняются. Конверт не больше 65 536 байтов, подписываемое представление сообщения не больше 24 576 байтов.

9 Команда через два сервера и два владельца

Signer подписывает назначение и тело на стороне отправителя. Matrix-сервер `alpha.example` передаёт событие серверу `beta.example` в разрешённую комнату; оба имени вымышлены для примера. Получатель проверяет конверт по закреплённому корню владельца отправителя, независимо от сервера доставки. Подмена содержимого или назначения проверку не проходит, но сервер может задержать, скрыть или повторить событие.

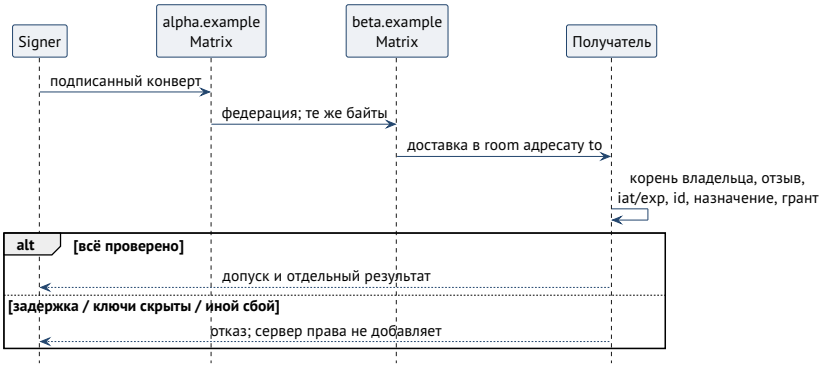


Рис. 9.1. Федерация перевозит событие, получатель устанавливает доверие

9.1 Доверие к другому владельцу

Корни и реестр другого владельца получатель закрепляет вне полосы, в защищённой конфигурации: владелец, ключи корней, роли и область. Аттестации другого владельца проверяются только против этого закрепления. Совпадение домена, приглашение в комнату и знакомое имя доверия не дают, а корень, пришедший в самом сообщении, ничего не закрепляет.

Устройство другого владельца в Matrix проходит ту же цепочку **device ← self-signing ← master ← аттестация корня оператора**, только корень берётся из закрепления этого владельца. Отзывы его ключей получатель принимает только подписанными закреплёнными корнями этого владельца, с теми же требованиями к возрасту и непрерывности, что и для своих снимков. Своих прав чужой корень не получает: полномочие на действие по-прежнему даёт только грант в домене владельца ресурса.

A0 удалённого сервера слабее доверия к своему серверу. При проверке по грантам A0 остаётся данными и критическую команду не обосновывает. Переходное доверие к своему серверу на чужой не распространяется без явной политики оператора.

Ключи носителя. Matrix cross-signing и списки устройств OMEMO приходят через федерацию. Аттестация корня не даёт чужому серверу добавить доверенное устройство. Сервер может не выдать ключи: тогда цепочку установить нельзя, и допуск закрыт. Сохранность закрытых ключей и корректность клиента остаются допущениями.

Время и повторы. Окно `iat` и `exp` сообщения и действия оператора не больше 120 секунд, и задержка федерации его не расширяет. Каждый проверяющий хранит свой журнал идентификаторов, а исполнитель хранит использованные `action_id`. Разные проверяющие общего хранилища повторов не образуют, поэтому однократный внешний эффект обеспечивает журнал исполнителя. Правка текста меняет `body_sha256` и требует новой подписи, а событие больше бюджета носителя отклоняется без усечения.

10 Уверенность и право действовать

Уверенность описывает свидетельство происхождения, а грант описывает разрешённое действие. Для допуска нужны **оба**, вместе со сроком, отзывом и совпадением цели. Высокий уровень уверенности не заменяет отсутствующего гранта.

Уровень	Свидетельство	Граница действия
A0	отправителя утверждает homeserver	при проверке по грантам данные; в переходном режиме только записанная зона названного человека
A1	проверенное E2EE-устройство человека или подпись аттестованного ключа агента	переписка; агентские сообщения остаются данными; критическое действие не разрешено
A2	ключ человека подписал точное содержимое, потребитель проверил его текущее полномочие	типизированное действие в пределах гранта; там, где нужно присутствие, A2 недостаточно
A3	квалифицированный провайдер показал точное действие и потребовал присутствия	расширяющие действия: выдача грантов, привязки, допуски; другие критические действия по политике гранта

10.1 Как проходит проверка гранта

Грант (`grant/1`) записан в закрытом формате CCJ-1 и называет ключи, действия, цели и временное окно. Ролей, счётчиков и языка правил в нём нет. Цепочка грантов не длиннее четырёх звеньев, и каждое дочернее звено только сужает родительское. Требование `min_assurance` задаёт уровень и признак присутствия, и дочерний грант может его только ужесточить.

Ворота доверия находят зарегистрированное действие и конкретную цель. Словарь действий закрыт: `credential.lease`, `agent.spawn` и `message.deliver`. Затем они проходят цепочку до закреплённого корня грантов: подписи издателей, получателей, сроки, окна и отзывы каждого звена. Ключ агента издателем гранта не бывает. Ключ сервиса грантов выдаёт гранты только под родительским грантом, адресованным ему и покрывающим дочерний. После этого ворота сверяют установленный уровень запроса с требованием листа. Детерминированные ворота не спрашивают модель, достаточно ли убедительно сообщение, а дополнительная проверка может только сузить допуск.

Исполнитель повторяет проверку текущего полномочия на границе эффекта. Подтверждение привязано к точному действию и на следующие реплики разговора не распространяется.

Пример: человеку разрешено читать отчёты, и он присылает A3-подтверждение удаления. Уверенность достаточна, полномочия нет, и ворота отказывают. Обратный случай: грант на удаление есть, но пришла обычная реплика из Element уровня A1, и ворота тоже отказывают.

Аппаратное подтверждение на Mac и FIDO-ключе становится A3 только после квалификации своего провайдера. Сам факт аппаратной подписи эквивалентности телефонному A3 не устанавливает.

11 Как устроен Signer

Общее ядро Signer строит и проверяет конверты, цепочки аттестаций и гранты, а CLI `a2a` служит одним из его интерфейсов. Адаптеры носителей подключаются по очереди: сначала Matrix, затем XMPP. Синхронизатор получает снимки отзывов, записи свежести и голову свидетеля, и детерминированные ворота выносят вердикт по ним; в сеть в момент вердикта ходит только онлайн-проверка критического действия. Приложения macOS и iOS встраивают ядро как framework на Relux Swift и Tuist, а секрет остаётся за провайдером ключа в Secure Enclave.

11.1 Открытые вопросы

Часть	Что осталось	Статус
Формат `swarm-envelope/2`, срез v0	Спецификация и офлайн-библиотека проверки; потребителей нет.	спецификация
Реестр и свидетель на узле доверия	Размещение, роли ключей и процесс подписи решены; службы нет.	решено
Отдельная запись свежести	Выбор `freshness/1` вместо переиздания снимка.	решено
Формат и проверка `freshness/1`	Спецификация прошла ревью и ждёт включения.	предложено
Квитанция свежести	Планируется; формата и решения в опубликованных спецификациях нет.	открыто
Сертификация агентских ключей	24 часа и доказательство публикации; подпись закрыта до реализации.	решено
Сервер свидетеля отката	Протокол предложен; транспорт и внешний хранитель открыты.	предложено
Провайдер A3	Цель: P-256 в Secure Enclave на iOS; не квалифицирован.	открыто
Завершение отзыва	Барьеры, доказательство истечения и протокол онлайн-проверки.	открыто
Корень и аккаунты носителя	Восстановление два из трёх, аттестации master и OMEMO, первое закрепление.	открыто
Носители и другие владельцы	Бюджет события Matrix, адаптер XMPP, обмен реестрами.	открыто
Другие записи	WebAuthn, Ed25519-ключи FIDO, историческая проверка, экстренная остановка.	открыто
Язык ядра	Рекомендован Rust/UniFFI с Matrix Rust SDK; ревью не завершено.	предложено